

Part A

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

```
In [ ]: data = pd.read_csv("customer_data.csv")
```

```
In [ ]: data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   CustomerID      500 non-null   int64
1   Age             500 non-null   int64
2   Income          500 non-null   int64
3   SpendingScore   500 non-null   int64
4   Gender          500 non-null   object
5   Profession       500 non-null   object
6   MaritalStatus   500 non-null   object
dtypes: int64(4), object(3)
memory usage: 27.5+ KB
```

```
In [ ]: data.head(10)
```

```
Out[ ]:
```

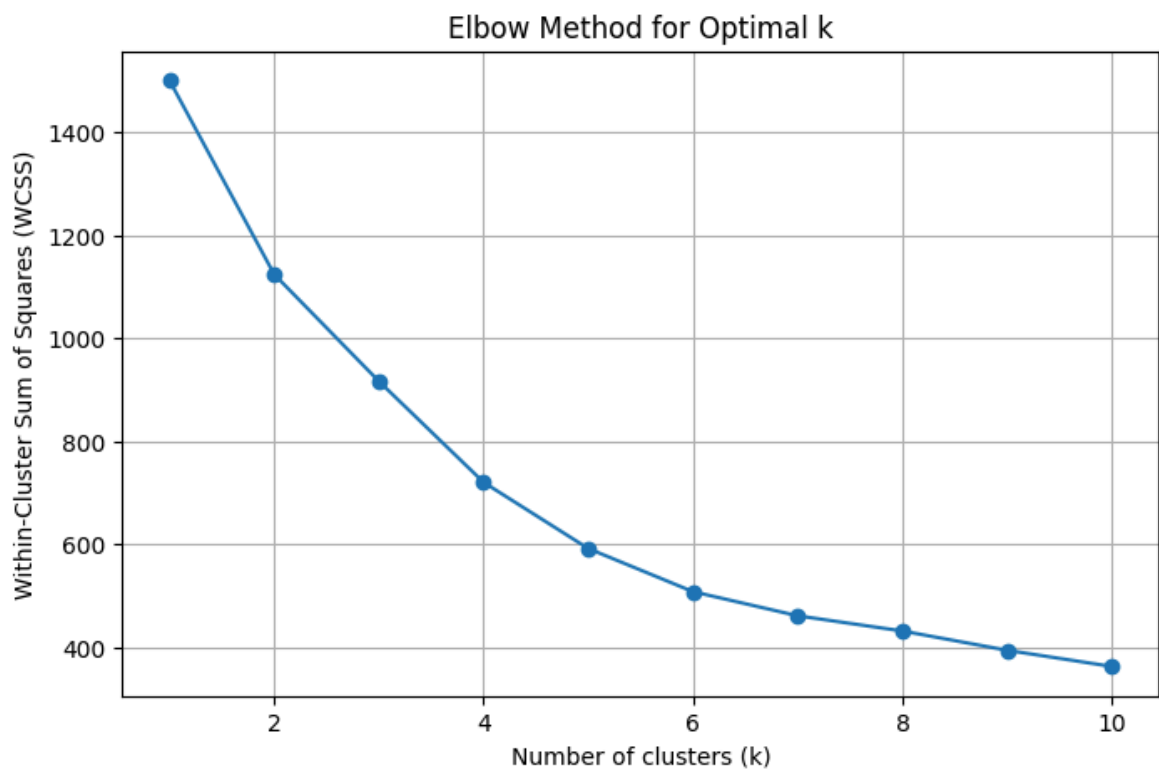
	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single
5	6	25	56074	26	Female	Teacher	Single
6	7	38	31086	29	Male	Doctor	Married
7	8	56	63767	21	Female	Student	Married
8	9	36	81832	11	Male	Lawyer	Married
9	10	40	65486	40	Female	Artist	Single

```
In [ ]: features = ['Age', 'Income', 'SpendingScore']
X = data[features]
```

```
In [ ]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [ ]: wcss = []
k_range = range(1, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```

```
In [ ]: plt.figure(figsize=(8, 5))
plt.plot(k_range, wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.grid(True)
plt.show()
```



```
In [ ]: k = 4
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(X_scaled)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

```
In [ ]: clusters = kmeans.fit_predict(X_scaled)
data['Cluster'] = clusters
```

```
In [ ]: for i in range(4):
    print(f"\n ♦ Cluster {i} sample data:")
    print(data[data['Cluster'] == i].head())
```

✚ Cluster 0 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus	\
3	4	32	40200	53	Male	Lawyer	Single	
5	6	25	56074	26	Female	Teacher	Single	
6	7	38	31086	29	Male	Doctor	Married	
10	11	28	27209	11	Male	Lawyer	Married	
11	12	28	42736	36	Female	Artist	Married	

	Cluster
3	0
5	0
6	0
10	0
11	0

✚ Cluster 1 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus	\
4	5	60	76481	36	Female	Student	Single	
7	8	56	63767	21	Female	Student	Married	
8	9	36	81832	11	Male	Lawyer	Married	
9	10	40	65486	40	Female	Artist	Single	
13	14	70	39384	39	Male	Teacher	Married	

	Cluster
4	1
7	1
8	1
9	1
13	1

✚ Cluster 2 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus	\
12	13	41	69003	59	Male	Student	Single	
14	15	53	56657	99	Female	Lawyer	Married	
20	21	19	46211	69	Female	Doctor	Single	
25	26	19	51961	83	Female	Doctor	Single	
26	27	38	71619	99	Female	Engineer	Married	

	Cluster
12	2
14	2
20	2
25	2
26	2

✚ Cluster 3 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus	\
0	1	56	31995	54	Male	Teacher	Single	
1	2	69	44982	65	Male	Artist	Married	
2	3	46	42875	51	Male	Lawyer	Married	
16	17	41	36096	98	Female	Teacher	Married	
21	22	41	31283	56	Female	Doctor	Single	

	Cluster
0	3
1	3
2	3
16	3
21	3

In []:

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np

# Example cluster names - adjust based on your analysis
cluster_names = {
    0: 'Young Spenders',
    1: 'High Income - High Spending',
    2: 'Savers',
    3: 'Low Income - Low Spending'
}

colors = ['blue', 'green', 'orange', 'purple']
k = len(np.unique(clusters))

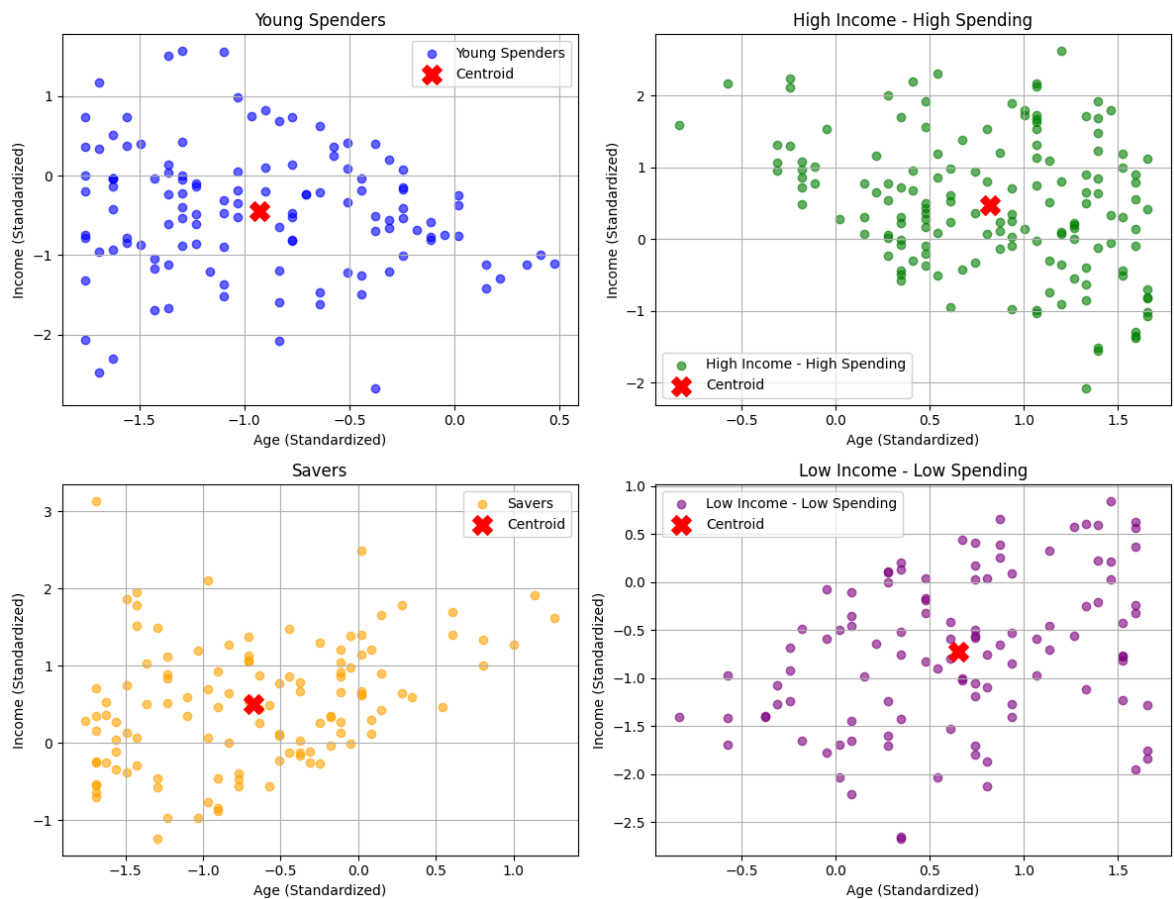
# 2x2 grid setup
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for i in range(k):
    ax = axes[i]
    ax.scatter(X_scaled[clusters == i, 0], X_scaled[clusters == i, 1],
               color=colors[i], label=cluster_names[i], alpha=0.6)
    ax.scatter(centroids[i, 0], centroids[i, 1], marker='X', s=200, c='red', label=cluster_names[i])
    ax.set_title(f'{cluster_names[i]}')
    ax.set_xlabel('Age (Standardized)')
    ax.set_ylabel('Income (Standardized)')
    ax.legend()
    ax.grid(True)

# Hide extra axes if clusters < 4
for j in range(k, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle('Customer Segmentation - Cluster Wise View with Names', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Customer Segmentation - Cluster Wise View with Names



In []:

Part 2

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
In [ ]: data = pd.read_csv("customer_data.csv")
data
```

Out []:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single
...	...	...	...	...	...	...	...
495	496	43	65215	64	Female	Lawyer	Married
496	497	70	22386	82	Male	Artist	Married
497	498	40	30806	95	Male	Teacher	Single
498	499	26	40627	38	Female	Student	Married
499	500	29	50391	32	Male	Engineer	Married

500 rows × 7 columns

In []: `data.head()`

Out []:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single

In []: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      500 non-null   int64
1   Age             500 non-null   int64
2   Income          500 non-null   int64
3   SpendingScore   500 non-null   int64
4   Gender          500 non-null   object
5   Profession       500 non-null   object
6   MaritalStatus   500 non-null   object
dtypes: int64(4), object(3)
memory usage: 27.5+ KB
```

In []: `missing_values = data.isnull().sum()`
`missing_values`

```
Out[ ]:      0
CustomerID  0
Age         0
Income      0
SpendingScore 0
Gender      0
Profession  0
MaritalStatus 0
```

dtype: int64

```
In [ ]: data_cleaned = data.drop(columns=["CustomerID"])
data_cleaned
```

```
Out[ ]:      Age  Income  SpendingScore  Gender  Profession  MaritalStatus
0      56    31995           54    Male    Teacher    Single
1      69    44982           65    Male     Artist    Married
2      46    42875           51    Male    Lawyer    Married
3      32    40200           53    Male    Lawyer    Single
4      60    76481           36  Female    Student    Single
...     ...      ...           ...     ...      ...      ...
495    43    65215           64  Female    Lawyer    Married
496    70    22386           82    Male     Artist    Married
497    40    30806           95    Male    Teacher    Single
498    26    40627           38  Female    Student    Married
499    29    50391           32    Male    Engineer    Married
```

500 rows × 6 columns

```
In [ ]: categorical_cols = ['Gender', 'Profession', 'MaritalStatus']
data_encoded = pd.get_dummies(data_cleaned, columns=categorical_cols)
```

```
In [ ]: data_encoded = data_encoded.dropna()
data_encoded
```

Out[]:

	Age	Income	SpendingScore	Gender_Female	Gender_Male	Profession_Artist	Pro
0	56	31995	54	False	True	False	
1	69	44982	65	False	True	True	
2	46	42875	51	False	True	False	
3	32	40200	53	False	True	False	
4	60	76481	36	True	False	False	
...	...	...	...	...	...	...	...
495	43	65215	64	True	False	False	
496	70	22386	82	False	True	True	
497	40	30806	95	False	True	False	
498	26	40627	38	True	False	False	
499	29	50391	32	False	True	False	

500 rows × 13 columns



```
In [ ]: kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(data_encoded[['Age', 'Income']])
data_encoded['Segment'] = kmeans.labels_
```

```
In [ ]: data['Segment'] = data_encoded['Segment']
```

```
In [ ]: plt.figure(figsize=(10, 6))
plt.scatter(data['Age'], data['Income'], c=data['Segment'], cmap='viridis', s=50)
plt.title('Customer Segmentation Based on Age and Income')
plt.xlabel('Age')
plt.ylabel('Income')
plt.colorbar(label='Segment')
plt.grid(True)
plt.show()
```

